

08 — Frontend Development Guidelines

Product: Endpoint IQ **Document:** Frontend Development Guidelines

Practical, enforceable rules for anyone (human or AI coding agent) implementing the Endpoint IQ frontend.

1. React Conventions

- Functional components + hooks only; no class components.
- One component per file; file name matches component name.
- Keep page components thin — they compose feature components and hooks, they don't fetch/transform data inline.
- Co-locate a component's styles, tests, and types with the component when they're not shared elsewhere.

2. TypeScript Conventions

- `strict: true` in `tsconfig.json`. No implicit any.
- Prefer interface for object shapes that model domain entities (matches 06-Frontend-State-and-Data-Models.md); type for unions, aliases, and utility compositions.
- No any — use unknown + narrowing for genuinely dynamic data (e.g. raw response bodies shown in JsonViewer).
- Export domain types from `types/`; never redefine a domain type inline in a component.

3. Component Naming

- PascalCase for components: `EndpointResultTable`, `AIAnalysisCard`.
- Suffix by role where it aids clarity: `*Form`, `*Table`, `*Card`, `*Modal`, `*Viewer`.

4. File Naming

- Component files: `ComponentName.tsx`.
- Hook files: `useHookName.ts`.
- Service files: `resourceService.ts` (camelCase, noun + "Service").
- Type files: `types.ts` within a feature, or `types/<domain>.ts` for shared types.

5. Folder Naming

- kebab-case for feature folders (`test-runs`, `ai-analysis`), matching route segments where relevant (see 04-Frontend-Pages-and-Routes.md).

6. Props Conventions

- Explicit interface `Props { ... }` per component, named `<ComponentName>Props`.
- No prop drilling more than 2 levels — lift to a hook or context if a value is needed deeper.
- Prefer composition (children/slots) over large boolean-flag prop lists for variant behavior.

7. Hooks

- One responsibility per hook (useTestRunProgress, useEndpointList).
- Data-fetching hooks wrap the server-state library (React Query or equivalent) — components never call services/* directly, only through a hook.
- Shared cross-feature hooks live in hooks/; feature-specific hooks stay in features/*/hooks/.

8. API Service Conventions

- One service module per backend domain (see 05-Frontend-API-Integration.md §17).
- Services return typed domain objects (06-...), never raw Axios/fetch response objects.
- Every service function has a single, clear responsibility (getProject, startTestRun) — no multi-purpose “god” functions.
- Any endpoint marked **Proposed API Contract** in 05-... must carry a // TODO: confirm with backend comment in the service file until verified.

9. State Management

- Server state → data-fetching library cache only (see 06-Frontend-State-and-Data-Models.md §6). Never duplicate server data into a global store “for convenience.”
- Global stores reserved for: auth/session flag, toast queue, theme/UI-shell state.
- Local component state for anything else, including form drafts and table filters.

10. Form Handling

- Client-side validation runs before submit for fast feedback; server validation errors (05-... §10) always take precedence and are mapped onto the same fields.
- Credential forms (username/password) never pre-fill or echo back a previously saved password — see 06-... Credential type (write-only password).
- Disable submit buttons while a request is in flight; show inline field errors, not just a toast, for validation failures.

11. Error Handling

- All API errors flow through the shared normalizer (05-... §7-8) before reaching UI code — no component should inspect raw HTTP status codes itself.
- Distinguish, at the type level, between a frontend/platform error and a target-API test failure (05-... §9) — these must never share the same error-rendering path.
- Every page has an error boundary or explicit error state (03-... §25); no unhandled promise rejections in feature code.

12. Loading States

- Every data-fetching hook exposes isLoading/isError/data (or equivalent) and every consuming component handles all three explicitly — no components that assume data is always present.
- Use skeletons for initial loads, inline spinners for actions, per 03-UI-UX-Specification.md §22.

13. Reusable Components

- Before building a new component, check components/ for an existing primitive (Table, Modal, Badge, etc.) — do not create a second one-off version.
- Any UI pattern used in 2+ features must be promoted to components/, not duplicated per-feature.

14. Styling Conventions

- Use design tokens (colors, spacing) from a central source (see 03-UI-UX-Specification.md Open Decisions on tokens) rather than hardcoded hex/px values scattered through components.
- Status colors (Success/Failed/Warning/Running/Pending/Critical/Al) are defined once and consumed via the shared StatusPill component — never re-implemented ad hoc.

Open Decision: Specific styling technology (CSS Modules, Tailwind, styled-components) is not defined in the source architecture document. Choose one and apply it consistently; do not mix approaches within the codebase.

15. Accessibility

- Every interactive element is keyboard-reachable and has a visible focus state.
- Status conveyed by color always has a text/icon fallback (03-... §28).
- Live-updating regions (test run progress) use `aria-live="polite"`.

16. Security

- Never store JWTs, session tokens, or credentials in `localStorage/sessionStorage` — this is a hard rule inherited directly from the architecture’s security model (§6), not a style preference.
- Password inputs are always masked; never log credential values to the console, even in development.
- Sanitize/escape any target-API response content rendered in `JsonViewer/CodeBlock` to prevent it from executing as HTML (responses come from a system under test and must be treated as untrusted data).
- Respect the “AI output is not automatically trusted” model in the UI — never auto-apply an AI recommendation as if it were a verified fix.

17. Environment Variables

- All frontend env vars are prefixed per the build tool’s convention (e.g. `VITE_`) and documented in `.env.example`.
- No secrets in frontend env vars — the frontend only ever needs public base URLs (see 02-... §13).

18. Code Quality

- No unused exports, no commented-out dead code committed.
- Prefer small, composable functions/components over large multi-purpose ones.
- Run linting/type-checking as a pre-commit or CI gate.

19. Avoiding Duplicate Code

- Shared formatting/parsing logic (status→color mapping, date formatting, diffing) lives once in `utils/`, imported everywhere it's needed.
- Before adding a new API error-handling branch, check the shared normalizer (05-... §7) first — don't re-implement error classification per-feature.

20. Testing

- Aligns with the architecture's overall testing target (§12 Implementation Checklist: "target 90%+ coverage before release," Playwright for e2e).
- Unit tests for `utils` and `hooks`; component tests for feature components with meaningful logic (forms, tables with filtering); e2e tests (Playwright) for the critical path: create project → import spec → configure role/credential → start run → view results → view AI analysis → download report.
- Mock the API service layer in component/unit tests; reserve real network calls for e2e tests against a test backend.

21. Performance

- Virtualize or paginate long tables (endpoint list, results table) once data sets exceed roughly a few hundred rows (see Open Decision on pagination in 05-... §15).
- Avoid re-fetching unchanged data — rely on the server-state cache's built-in deduplication/staleness rules rather than manual `useEffect` fetch chains.
- Debounce search/filter inputs on large tables.

22. Git Conventions

- Feature branches named `feature/<short-description>`, fixes `fix/<short-description>`.
- Commit messages describe the change and, where relevant, which workflow stage/feature it touches (e.g. `feat(test-runs): add live progress WS hook`).
- PRs should reference the relevant doc section (e.g. "implements 04-... /projects/:projectId/test-runs/:runId") so reviewers can cross-check against the spec.

Cross-references: 02-Frontend-Architecture.md (structure these rules apply to), 06-Frontend-State-and-Data-Models.md (typing rules), 09-Frontend-Development-Guide.md (how these guidelines fit the build sequence).